



13. Desafio de projeto: Otimizando o sistema bancário com funções Python

☑ Dúvidas?	☐
☑ Exercícios a entregar?	☒
📁 Módulo	2 - Dominando funções e boas práticas em Python
☰ Tipo de aula	Leitura Tarefa de programação Vídeo
🔔 Fora do prazo	🕒 Fora do prazo
☰ Links e mídias	Pasta "Aula 13"
🕒 Última atualização	@18 de novembro de 2025 06:03
⚙ Status	Not started
📅 Data início	@18/11/2025
⌚ Esforço	Difícil

[Anotações](#)

[Descrição](#)

[Vídeo 1 - Apresentação do desafio](#)

[Objetivo geral](#)

[Desafio](#)

[Vídeo 2 - Resolução do desafio](#)

[Exercícios](#)

[Arquivos](#)

Anotações

Descrição

Neste desafio, você terá a oportunidade de otimizar um Sistema bancário previamente desenvolvido com o uso de funções Python. O objetivo é aprimorar a estrutura e a eficiência do sistema, implementando as operações de depósito, saque e extrato em funções específicas. Você terá a chance de refatorar o código existente, dividindo-o em funções reutilizáveis, facilitando a manutenção e o entendimento do sistema como um todo. Prepare-se para aplicar conceitos avançados de programação e demonstrar sua habilidade em criar soluções mais elegantes e eficientes utilizando Python.

Vídeo 1 - Apresentação do desafio

Objetivo geral

Separar as funções existentes de saque, depósito e extrato em funções. Criar duas novas funções: cadastrar usuário (cliente) e cadastrar conta bancária.

Desafio

Precisamos deixar o código mais modularizado, para isso criaremos funções para as operações existentes: sacar, depositar e visualizar histórico (extrato). Além disso, para a versão 2 do sistema, precisamos criar duas novas funções: criar usuário (cliente do banco) e criar conta corrente (vincular com usuário).

▼ Separação em funções

Devemos criar funções para todas as operações do sistema. Para exercitar tudo o que aprendemos neste módulo, cada função terá uma regra para passar os parâmetros. O retorno e a forma como as funções serão

chamadas podem ser definidos pelo aluno(a), da forma que ele(a) achar melhor.

- A ideia é exercitar o uso de passar argumentos/parâmetros por posição e por nome.
- A função **saque** deve receber os parâmetros apenas por nome. Sugestões de argumentos: saldo, valor, extrato, limite, numero_saques, limite_saques. Sugestões de retorno: saldo e extrato.
- A função **depósito** deve receber os parâmetros apenas por posição. Sugestão de argumentos: saldo, valor, extrato. Sugestão de retorno: saldo e extrato.
- A função **extrato** deve receber os parâmetros por posição e nome. Argumentos posicionais: saldo, argumentos nomeados: extrato.
- As novas funções serão **criar usuário** e **criar conta corrente**. Fique à vontade para adicionar mais funções, como **listar contas**.

▼ Novas funções

- A função **criar usuário** deve armazenar os usuários em uma lista. Um usuário é composto por: **nome**, **data de nascimento**, **CPF** e **endereço**. O endereço é `str`, no formato "**logradouro, número - bairro - cidade/sigla estado**". Só devem ser armazenados os números do CPF (que também é `str`), e dois usuários não podem ter o mesmo CPF.
- A função **criar conta corrente** deve armazenar contas em uma lista. Uma conta é composta por: **agência**, **número da conta** e **usuário**. O número da conta é sequencial, começando em 1. O número da agência é fixo: "0001". Um usuário pode ter mais de uma conta, mas uma conta deve pertencer a somente um usuário.

▼ Dica

- Para vincular um usuário a uma conta, filtre toda a lista de usuários buscando pelo número do CPF.
- Se o CPF não existe, a conta não pode ser criada.

Vídeo 2 - Resolução do desafio

Exercícios

Arquivos

Pasta "Aula 13"